Process & Decision Documentation

Side Quests

Focus on:

This game aims to evoke a meditative experience, allowing players to relax through calm and peaceful exploration. The camera automatically tracks a smoothly flying airplane, endlessly flying to the right. A deep parallax system creates a 3D illusion through three layers of twinkling star. As the airplane flies, birds will spawn randomly.

If GenAI was used (keep each response as brief as possible):

Date Used: Feb 21, 2026

Tool Disclosure: Gemini 3

Purpose of Use: Code generation, refactoring math logic, and CSS styling for a p5.js sketch.

Summary of Interaction: GenAI generated updated JavaScript and CSS code, transforming a limited top-down perspective game into an infinite scrolling parallax effect experience. It generated a starry sky background, airplane animations, and hidden interactive objects.

Human Decision Point(s): Chose the starry sky/airplane theme, change from geometric runes to birds, refined the birds' aesthetic to be minimalist, and requested specific CSS framing.

Integrity & Verification Note: Reviewed the AI-generated p5.js code to ensure the game renders correctly.

Scope of GenAI Use: GenAI wrote the procedural logic, parallax math, and CSS layout. It did not define the core conceptual shift, pacing requirements, or aesthetic choices (e.g., minimalist birds, framing), which were human directed.

Limitations or Misfires: The AI initially generated complex shapes and glowing auras that clashed with the requested minimalist, meditative aesthetic. It required iterative prompting to strip away unnecessary visual details.

Summary of Process (Human + Tool)

I provided the creative vision, pacing goals, and aesthetic adjustments. The AI handled the technical implementation, translating those ideas into functional p5.js algorithms.

Decision Points & Trade-offs

Minimalism vs. Detail: Traded detailed bird shapes and glowing effects for primitive geometric silhouettes to maintain the calming, minimalist tone.

Verification & Judgement

Changes were evaluated primarily through iterative visual playtesting. After every code injection, I tested the sketch in the browser to check the aesthetic against the initial goal of creating a "reflective" experience.

Limitations, Dead Ends, or Open Questions

The AI initially generated complex shapes and glowing auras that clashed with the requested minimalist, meditative aesthetic. It required iterative prompting to strip away unnecessary visual details.

Appendix

Appendix: GenAI Transcript

Tool Used: Gemini **Date:** Feb 21, 2026

Prompt 1: Create a reflective or meditative camera experience that scrolls through a world larger than the screen. Use pacing and motion to evoke emotion. Bonus: Hide small interactive symbols or objects for the camera to “discover.

[uploaded index.html, jsconfig.json, sketch.js, style.css]

Gemini answer:

This is a wonderful direction to take a sketch! To shift the mood from a rigid, player-driven top-down game to a reflective and meditative experience, we need to focus heavily on **pacing, easing, and atmosphere**.
Here is how we can transform your current code:

1. **The Camera:** Instead of snapping to player input, the camera will now wander autonomously using 2D Perlin noise. This creates a slow, organic drifting motion—like floating down a river or looking through a telescope.
2. **The Environment:** The harsh grid lines and blocky obstacles have been replaced with a deep, calming background, soft parallax "dust" or stars, and faint glowing nebulas.
3. **The Discovery (Bonus):** I've scattered invisible symbols throughout the vast world. As the camera slowly drifts, you can use your mouse as a "focal point." When your mouse gets close to a hidden symbol, it will gently bloom into a glowing shape, lingering slightly even after you move away.

Updated sketch.js

Replace the entire contents of your sketch.js file with this code:

JavaScript

```javascript
/*
Reflective Camera Experience

Goal:
- Slower pacing and motion to evoke a calming emotion.
- Autonomous camera that drifts through a vast world.
- Hidden interactive symbols to discover with the mouse.
*/

let cam = { x: 0, y: 0 };
const W = 3000, H = 2000;       // Vast world size
const viewW = 800, viewH = 480; // Viewport size

let dust = [];
let symbols = [];
let t = 0; // Time variable for Perlin noise

function setup() {
  createCanvas(viewW, viewH);

  // Initialize background floating particles (parallax layer)
  for (let i = 0; i < 200; i++) {
    dust.push({
      x: random(W * 1.5),
      y: random(H * 1.5),
      size: random(1, 4),
      speed: random(0.05, 0.2)
    });
  }

  // Initialize hidden interactive symbols
  for (let i = 0; i < 45; i++) {
    symbols.push({
      x: random(200, W - 200),
      y: random(200, H - 200),
      activation: 0, // Scales from 0 (hidden) to 1 (glowing)
      discovered: false
```

```
    });
  }
}

function draw() {
  // Deep, calming night/void atmosphere
  background(15, 20, 35);

  // --- 1. Meditative Camera Drift ---
  t += 0.0015; // Extremely slow pacing

  // Use Perlin noise to smoothly and unpredictably pan the camera.
  // We map the noise to the boundaries so the camera never leaves the world.
  cam.x = map(noise(t), 0.2, 0.8, 0, W - width, true);
  cam.y = map(noise(t + 500), 0.2, 0.8, 0, H - height, true);

  // --- 2. Parallax Background ---
  push();
  translate(-cam.x * 0.3, -cam.y * 0.3); // Moves slower to create depth
  noStroke();
  fill(255, 255, 255, 60);
  for (let d of dust) {
    d.y -= d.speed; // Particles slowly float upwards
    if (d.y < -100) d.y = H * 1.5;
    circle(d.x, d.y, d.size);
  }
  pop();

  // --- 3. Main World Layer ---
  push();
  translate(-cam.x, -cam.y);

  // Draw soft, ambient background shapes (nebulas/auras)
  noStroke();
  for (let i = 0; i < 15; i++) {
    let cx = (noise(i) * W * 2) % W;
    let cy = (noise(i + 100) * H * 2) % H;
    let r = noise(i + 200) * 400 + 100;

    // Layered circles for a soft gradient effect
    fill(40, 60, 100, 8);
    circle(cx, cy, r);
    circle(cx, cy, r * 0.7);
    circle(cx, cy, r * 0.4);
  }

  // Calculate mouse position relative to the moving world
  let worldMouseX = mouseX + cam.x;
  let worldMouseY = mouseY + cam.y;

  // Render and update interactive symbols
  for (let sym of symbols) {
    // Check distance from the mouse's focal point
```

```
    let d = dist(worldMouseX, worldMouseY, sym.x, sym.y);

    // Smoothly fade in if the mouse gets close
    if (d < 120) {
      sym.activation = lerp(sym.activation, 1, 0.04);
      sym.discovered = true;
    } else {
      // Slowly fade out, but retain a faint memory/glow once discovered
      let targetAlpha = sym.discovered ? 0.15 : 0;
      sym.activation = lerp(sym.activation, targetAlpha, 0.02);
    }

    // Only draw the symbol if it's currently visible
    if (sym.activation > 0.01) {
      push();
      translate(sym.x, sym.y);
      rotate(frameCount * 0.005 + sym.x); // Very slow spin

      let alpha = sym.activation * 255;
      let size = 10 + sym.activation * 20;

      // Outer Glow
      fill(255, 220, 150, alpha * 0.2);
      circle(0, 0, size * 2.5);

      // Core Geometric Shape (Abstract Star)
      fill(255, 240, 200, alpha);
      for(let j = 0; j < 4; j++) {
        rotate(PI / 4);
        rectMode(CENTER);
        rect(0, 0, size, size * 0.15, 3);
      }
      pop();
    }
  }
  pop();

  // --- 4. Vignette & UI Overlay ---
  // Soft shadows around the edges of the canvas to frame the view
  noFill();
  for (let i = 0; i < 40; i++) {
    stroke(0, 0, 0, i * 2.5);
    rect(i, i, width - i * 2, height - i * 2);
  }

  // Minimalist instructions
  fill(255, 255, 255, 60);
  noStroke();
  textFont("sans-serif");
  textSize(12);
  textAlign(RIGHT);
  text("drift and uncover", width - 20, height - 20);
}
```

Why this works:

- **The noise() function:** In the original, you used precise linear calculations (constrain(), WASD input) for control. By using Perlin noise instead, we get smooth, continuous, and organic camera transitions that feel alive but out of the user's immediate control.
- **lerp() for Easing:** Instead of the symbols immediately turning "on" or "off", the lerp() (linear interpolation) function allows the alpha channel to gently glide up and down, matching the meditative pacing of the sketch.

Would you like me to help you add a generative soundscape to this using the p5.sound.min.js library you already have linked in your HTML?

Prompt 2: I want a starry sky background and an airplane that automatically flies forward.

Gemini answer:

This is a great shift in direction! Moving from a drifting, abstract world to an infinite flight creates a fantastic sense of forward momentum while keeping that calming, atmospheric vibe.

To achieve an endlessly flying airplane and a vast starry sky, we can use a neat trick: **modulo wrapping**. Instead of generating millions of stars for an infinitely wide world, we only generate a screen's worth of stars. As the camera moves forward, we wrap the stars that fall off the left side of the screen back over to the right side. Combined with different movement speeds (parallax), it creates the illusion of infinite 3D depth.

Updated sketch.js

Replace the contents of your sketch.js file with this code:

JavaScript

```
/*
Endless Starry Flight

Goal:
- Infinite starry sky background with parallax depth.
- Airplane automatically flying forward.
- Camera continuously tracks the airplane.
*/

let plane = { x: 100, y: 240, speed: 3.5 };
let cam = { x: 0, y: 0 };
const viewW = 800, viewH = 480;

// Arrays to hold different layers of stars for parallax
let backStars = [];
let midStars = [];
let frontStars = [];
```

```
function setup() {
  createCanvas(viewW, viewH);

  // Generate a screen's worth of stars for each depth layer
  for (let i = 0; i < 150; i++) backStars.push(createStar(1, 2));
  for (let i = 0; i < 100; i++) midStars.push(createStar(2, 3));
  for (let i = 0; i < 40; i++) frontStars.push(createStar(3, 5));
}

// Helper function to create a random star
function createStar(minSize, maxSize) {
  return {
    x: random(0, viewW),
    y: random(0, viewH),
    size: random(minSize, maxSize),
    twinkleOffset: random(TWO_PI) // Randomizes the twinkle timing
  };
}

function draw() {
  // Deep night sky gradient background
  background(12, 16, 35);

  // --- 1. Update Airplane & Camera ---
  plane.x += plane.speed;
  // Add a gentle vertical bobbing motion to the plane using a sine wave
  plane.y = viewH / 2 + sin(frameCount * 0.04) * 25;

  // The camera follows the plane, keeping it fixed on the left side of the view
  cam.x = plane.x - 200;

  // --- 2. Draw Star Layers (Parallax) ---
  // The second parameter is the parallax speed (closer = moves faster)
  drawStars(backStars, 0.1, 100);
  drawStars(midStars, 0.4, 180);
  drawStars(frontStars, 0.8, 255);

  // --- 3. Draw the Airplane ---
  push();
  // We translate to the plane's position relative to the camera
  translate(plane.x - cam.x, plane.y);

  noStroke();

  // Back wing
  fill(160, 170, 190);
  triangle(-10, 0, 15, 0, -15, -35);

  // Fuselage (Body)
  fill(220, 230, 240);
  ellipse(0, 0, 90, 22);
```

```
  // Tail
  fill(190, 200, 215);
  triangle(-30, 0, -45, -25, -20, -5);

  // Front wing
  fill(200, 210, 225);
  triangle(-15, 5, 25, 5, -25, 45);

  // Cockpit window glowing slightly
  fill(150, 220, 255, 200);
  arc(25, -4, 25, 16, PI, TWO_PI);

  pop();
}

// Function to handle the infinite scrolling and rendering of stars
function drawStars(starArray, parallaxFactor, maxAlpha) {
  noStroke();
  for (let s of starArray) {
    // Calculate the star's X position on screen.
    // The modulo (%) operator wraps the star back to the right side when it goes off the left.
    let screenX = (s.x - (cam.x * parallaxFactor)) % viewW;

    // JavaScript modulo can return negative numbers, so we correct it to wrap properly
    if (screenX < 0) screenX += viewW;
    let screenY = s.y;

    // Calculate a twinkling effect using a sine wave
    let alpha = map(sin(frameCount * 0.05 + s.twinkleOffset), -1, 1, maxAlpha * 0.2, maxAlpha);
    fill(255, 255, 255, alpha);

    circle(screenX, screenY, s.size);
  }
}
```

How this works:

- **The `drawStars()` function:** The line `(s.x - (cam.x * parallaxFactor)) % viewW` is doing the heavy lifting. As `cam.x` grows infinitely larger, subtracting it moves the stars to the left. The modulo (`%`) forces that shrinking number to stay within the bounds of your screen width (`viewW`).
- **The `sin()` function:** I added a `sin()` wave to the airplane's `y` coordinate to give it a smooth, gentle bobbing motion, and another `sin()` wave to the stars' transparency to make them twinkle.

Would you like me to add a particle system so the airplane leaves a fading contrail or glowing engine exhaust as it flies?

Prompt 3: Hide small interactive symbols or objects for the camera to “discover.”

Gemini answer:

This is a perfect addition! Bringing that "discovery" mechanic into an infinite flight changes the technical approach slightly. Because the world is endless, we can't just place 45 symbols at the start—we'd eventually fly past all of them and run out.

Instead, we need to **procedurally generate** new hidden objects just off-screen ahead of the airplane, and delete the old ones once they fall far behind the camera. This ensures the sketch can run forever without slowing down your computer.
As the plane flies near these hidden coordinates, its proximity will "illuminate" them, revealing glowing geometric runes.

Updated sketch.js

Replace the code in your sketch.js file with this version:
JavaScript

```
/*
Endless Starry Flight with Hidden Discoveries

Goal:
- Infinite starry sky background with parallax depth.
- Airplane automatically flying forward.
- Hidden geometric runes procedurally spawn and light up as the plane passes near them.
*/

let plane = { x: 100, y: 240, speed: 3.5 };
let cam = { x: 0, y: 0 };
const viewW = 800, viewH = 480;

let backStars = [];
let midStars = [];
let frontStars = [];

// Array to hold our hidden objects and a tracker for when to spawn the next one
let symbols = [];
let nextSymbolX = 800;

function setup() {
  createCanvas(viewW, viewH);

  for (let i = 0; i < 150; i++) backStars.push(createStar(1, 2));
  for (let i = 0; i < 100; i++) midStars.push(createStar(2, 3));
  for (let i = 0; i < 40; i++) frontStars.push(createStar(3, 5));
}

function createStar(minSize, maxSize) {
  return {
    x: random(0, viewW),
    y: random(0, viewH),
    size: random(minSize, maxSize),
    twinkleOffset: random(TWO_PI)
  };
}
```

```
function draw() {
  background(12, 16, 35);

  // --- 1. Update Airplane & Camera ---
  plane.x += plane.speed;
  plane.y = viewH / 2 + sin(frameCount * 0.04) * 25;
  cam.x = plane.x - 200;

  // --- 2. Draw Star Layers ---
  drawStars(backStars, 0.1, 100);
  drawStars(midStars, 0.4, 180);
  drawStars(frontStars, 0.8, 255);

  // --- 3. Procedural Symbol Spawning & Discovery ---
  // If the camera view gets close to our target X, spawn a new hidden symbol ahead
  if (cam.x + viewW + 200 > nextSymbolX) {
    symbols.push({
      x: nextSymbolX,
      y: random(80, viewH - 80),
      activation: 0,
      discovered: false,
      rotation: random(TWO_PI),
      points: floor(random(3, 7)) // Random geometry (triangle, square, pentagon, etc.)
    });
    // Set the distance for the *next* symbol to spawn
    nextSymbolX += random(400, 1000);
  }

  // Iterate backwards through the array to safely remove off-screen symbols
  for (let i = symbols.length - 1; i >= 0; i--) {
    let sym = symbols[i];

    // Garbage collection: Remove symbols that are far behind the camera
    if (sym.x < cam.x - 200) {
      symbols.splice(i, 1);
      continue;
    }

    // Discovery Logic: Check the distance from the airplane to the symbol
    let d = dist(plane.x, plane.y, sym.x, sym.y);

    // If the plane is close, quickly light up the symbol
    if (d < 220) {
      sym.activation = lerp(sym.activation, 1, 0.06);
      sym.discovered = true;
    } else {
      // If the plane moves away, fade it to a lingering, faint memory glow
      let targetAlpha = sym.discovered ? 0.15 : 0;
      sym.activation = lerp(sym.activation, targetAlpha, 0.02);
    }

    // Only draw if it's currently visible
```

```
    if (sym.activation > 0.01) {
      push();
      // Translate from world coordinates to screen space
      translate(sym.x - cam.x, sym.y);
      rotate(sym.rotation + frameCount * 0.01); // Gentle spin

      let alpha = sym.activation * 255;
      let size = 10 + sym.activation * 15;

      // Draw glowing aura
      noStroke();
      fill(150, 220, 255, alpha * 0.2);
      circle(0, 0, size * 3);

      // Draw crisp geometric core
      stroke(200, 240, 255, alpha);
      strokeWeight(2);
      noFill();
      polygon(0, 0, size * 0.6, sym.points);

      pop();
    }
  }

  // --- 4. Draw the Airplane ---
  push();
  translate(plane.x - cam.x, plane.y);
  noStroke();

  fill(160, 170, 190); triangle(-10, 0, 15, 0, -15, -35); // Back wing
  fill(220, 230, 240); ellipse(0, 0, 90, 22);             // Body
  fill(190, 200, 215); triangle(-30, 0, -45, -25, -20, -5); // Tail
  fill(200, 210, 225); triangle(-15, 5, 25, 5, -25, 45);  // Front wing
  fill(150, 220, 255, 200); arc(25, -4, 25, 16, PI, TWO_PI); // Cockpit

  pop();
}

function drawStars(starArray, parallaxFactor, maxAlpha) {
  noStroke();
  for (let s of starArray) {
    let screenX = (s.x - (cam.x * parallaxFactor)) % viewW;
    if (screenX < 0) screenX += viewW;
    let screenY = s.y;

    let alpha = map(sin(frameCount * 0.05 + s.twinkleOffset), -1, 1, maxAlpha * 0.2, maxAlpha);
    fill(255, 255, 255, alpha);
    circle(screenX, screenY, s.size);
  }
}

// Helper function to draw dynamic geometric shapes
function polygon(x, y, radius, npoints) {
```

```
  let angle = TWO_PI / npoints;
  beginShape();
  for (let a = 0; a < TWO_PI; a += angle) {
    let sx = x + cos(a) * radius;
    let sy = y + sin(a) * radius;
    vertex(sx, sy);
  }
  endShape(CLOSE);
}
```

What's New:

- **Garbage Collection (`splice()`):** By iterating backward through the `symbols` array (`for (let i = symbols.length - 1; i >= 0; i--)`), we can safely delete shapes that the camera has flown past. This ensures your computer's memory doesn't get clogged up after a few minutes of flying.
- **The `polygon()` Function:** Rather than drawing the exact same symbol every time, I added a helper function that draws shapes based on points. When a symbol spawns, it is randomly assigned between 3 to 6 points (creating triangles, squares, pentagons, or hexagons), making the discoveries feel slightly unique.

Would you like me to help you map the `plane.y` coordinate to your mouse so you can actively fly the plane up and down to hunt for these symbols?

Prompt 4: Replace these geometric shapes with little birds.

Gemini answer:

This is a beautiful idea. Swapping cold geometric shapes for little birds instantly gives the sketch more life and turns it into a peaceful, storybook-like experience.

To make the birds feel alive, we won't just draw a static shape. We can use our trusty `sin()` wave function to animate their wings flapping up and down, and give them a slight forward flying speed so they are actively traveling through the sky with you.

Updated `sketch.js`

Replace the code in your `sketch.js` file with this version:

JavaScript

```
/*
Endless Starry Flight with Hidden Birds

Goal:
- Infinite starry sky background with parallax depth.
- Airplane automatically flying forward.
- Hidden flocks of birds spawn and illuminate as the plane passes near them.
*/

let plane = { x: 100, y: 240, speed: 3.5 };
let cam = { x: 0, y: 0 };
const viewW = 800, viewH = 480;
```

```
let backStars = [];
let midStars = [];
let frontStars = [];

// Array to hold our hidden birds and a tracker for when to spawn the next one
let birds = [];
let nextBirdX = 800;

function setup() {
  createCanvas(viewW, viewH);

  for (let i = 0; i < 150; i++) backStars.push(createStar(1, 2));
  for (let i = 0; i < 100; i++) midStars.push(createStar(2, 3));
  for (let i = 0; i < 40; i++) frontStars.push(createStar(3, 5));
}

function createStar(minSize, maxSize) {
  return {
    x: random(0, viewW),
    y: random(0, viewH),
    size: random(minSize, maxSize),
    twinkleOffset: random(TWO_PI)
  };
}

function draw() {
  background(12, 16, 35);

  // --- 1. Update Airplane & Camera ---
  plane.x += plane.speed;
  plane.y = viewH / 2 + sin(frameCount * 0.04) * 25;
  cam.x = plane.x - 200;

  // --- 2. Draw Star Layers ---
  drawStars(backStars, 0.1, 100);
  drawStars(midStars, 0.4, 180);
  drawStars(frontStars, 0.8, 255);

  // --- 3. Procedural Bird Spawning & Discovery ---
  // If the camera view gets close to our target X, spawn a new hidden bird ahead
  if (cam.x + viewW + 200 > nextBirdX) {
    birds.push({
      x: nextBirdX,
      y: random(80, viewH - 80),
      speed: random(1, 2),        // The bird's forward flight speed
      flapSpeed: random(0.3, 0.6),// How fast the wings flap
      flapOffset: random(TWO_PI), // Offsets the flap so they don't all flap in sync
      activation: 0,
      discovered: false
    });
    nextBirdX += random(300, 800);
  }
```

```
// Iterate backwards to safely update and remove off-screen birds
for (let i = birds.length - 1; i >= 0; i--) {
  let b = birds[i];

  // Birds fly forward autonomously
  b.x += b.speed;

  // Garbage collection: Remove birds that are far behind the camera
  if (b.x < cam.x - 200) {
    birds.splice(i, 1);
    continue;
  }

  // Discovery Logic
  let d = dist(plane.x, plane.y, b.x, b.y);

  if (d < 220) {
    b.activation = lerp(b.activation, 1, 0.06);
    b.discovered = true;
  } else {
    let targetAlpha = b.discovered ? 0.2 : 0;
    b.activation = lerp(b.activation, targetAlpha, 0.02);
  }

  // Only draw if the bird is currently visible
  if (b.activation > 0.01) {
    push();
    translate(b.x - cam.x, b.y);

    let alpha = b.activation * 255;

    // Draw soft glowing aura
    noStroke();
    fill(150, 220, 255, alpha * 0.2);
    circle(0, 0, 35);

    // Draw the Bird Silhouette
    fill(200, 240, 255, alpha);

    // Tail
    triangle(-6, 0, -12, -4, -12, 4);
    // Body
    ellipse(0, 0, 14, 8);
    // Head
    circle(6, -2, 8);
    // Beak
    triangle(8, -3, 13, -1, 8, 1);

    // Animated Flapping Wing
    // Maps the sine wave to a vertical position for the wing tip
    let wingY = sin(frameCount * b.flapSpeed + b.flapOffset) * 10;
    triangle(2, 0, -4, wingY, 6, wingY);
```

```
    pop();
  }
}

// --- 4. Draw the Airplane ---
push();
translate(plane.x - cam.x, plane.y);
noStroke();

fill(160, 170, 190); triangle(-10, 0, 15, 0, -15, -35); // Back wing
fill(220, 230, 240); ellipse(0, 0, 90, 22);             // Body
fill(190, 200, 215); triangle(-30, 0, -45, -25, -20, -5); // Tail
fill(200, 210, 225); triangle(-15, 5, 25, 5, -25, 45);  // Front wing
fill(150, 220, 255, 200); arc(25, -4, 25, 16, PI, TWO_PI); // Cockpit

pop();
}

function drawStars(starArray, parallaxFactor, maxAlpha) {
  noStroke();
  for (let s of starArray) {
    let screenX = (s.x - (cam.x * parallaxFactor)) % viewW;
    if (screenX < 0) screenX += viewW;
    let screenY = s.y;

    let alpha = map(sin(frameCount * 0.05 + s.twinkleOffset), -1, 1, maxAlpha * 0.2, maxAlpha);
    fill(255, 255, 255, alpha);
    circle(screenX, screenY, s.size);
  }
}
```

What changed:

- **Bird Physics:** The birds now have a `speed` variable. Because their speed (1 to 2) is slower than the plane's speed (3.5), the plane slowly overtakes them, but they are still actively moving.
- **Flapping Animation:** I added `flapSpeed` and `flapOffset` properties to the birds. When drawing the wing (the last `triangle()` in the bird drawing section), it uses a `sin()` wave to rapidly move the tip of the wing up and down between `-10` and `10` pixels.
- **Bird Anatomy:** Replaced the polygon helper function entirely with primitive shapes (`ellipse`, `circle`, and `triangle`) arranged to look like a small bird in flight from a side profile.

Would you like me to map the airplane's vertical position (`plane.y`) to your mouse so you can actively fly up and down to find the birds, rather than just waiting for them to cross your path?

Prompt 5: No need for the bird's tail and the blue circle around it.

Gemini answer:

That makes sense! Removing the glowing aura and the tail gives the birds a much cleaner, minimalist silhouette that fits perfectly with a quiet, meditative atmosphere.

Here is the updated code with those elements stripped out.

Updated sketch.js

Replace the code in your sketch.js file with this version:

JavaScript

```
/*
Endless Starry Flight with Minimalist Birds

Goal:
- Infinite starry sky background with parallax depth.
- Airplane automatically flying forward.
- Hidden minimalist birds spawn and illuminate as the plane passes near them.
*/

let plane = { x: 100, y: 240, speed: 3.5 };
let cam = { x: 0, y: 0 };
const viewW = 800, viewH = 480;

let backStars = [];
let midStars = [];
let frontStars = [];

// Array to hold our hidden birds and a tracker for when to spawn the next one
let birds = [];
let nextBirdX = 800;

function setup() {
  createCanvas(viewW, viewH);

  for (let i = 0; i < 150; i++) backStars.push(createStar(1, 2));
  for (let i = 0; i < 100; i++) midStars.push(createStar(2, 3));
  for (let i = 0; i < 40; i++) frontStars.push(createStar(3, 5));
}

function createStar(minSize, maxSize) {
  return {
    x: random(0, viewW),
    y: random(0, viewH),
    size: random(minSize, maxSize),
    twinkleOffset: random(TWO_PI)
  };
}
```

```
function draw() {
  background(12, 16, 35);

  // --- 1. Update Airplane & Camera ---
  plane.x += plane.speed;
  plane.y = viewH / 2 + sin(frameCount * 0.04) * 25;
  cam.x = plane.x - 200;

  // --- 2. Draw Star Layers ---
  drawStars(backStars, 0.1, 100);
  drawStars(midStars, 0.4, 180);
  drawStars(frontStars, 0.8, 255);

  // --- 3. Procedural Bird Spawning & Discovery ---
  if (cam.x + viewW + 200 > nextBirdX) {
    birds.push({
      x: nextBirdX,
      y: random(80, viewH - 80),
      speed: random(1, 2),
      flapSpeed: random(0.3, 0.6),
      flapOffset: random(TWO_PI),
      activation: 0,
      discovered: false
    });
    nextBirdX += random(300, 800);
  }

  // Iterate backwards to safely update and remove off-screen birds
  for (let i = birds.length - 1; i >= 0; i--) {
    let b = birds[i];

    // Birds fly forward autonomously
    b.x += b.speed;

    // Garbage collection: Remove birds that are far behind the camera
    if (b.x < cam.x - 200) {
      birds.splice(i, 1);
      continue;
    }

    // Discovery Logic
    let d = dist(plane.x, plane.y, b.x, b.y);

    if (d < 220) {
      b.activation = lerp(b.activation, 1, 0.06);
      b.discovered = true;
    } else {
      let targetAlpha = b.discovered ? 0.2 : 0;
      b.activation = lerp(b.activation, targetAlpha, 0.02);
    }

    // Only draw if the bird is currently visible
```

```
    if (b.activation > 0.01) {
      push();
      translate(b.x - cam.x, b.y);

      let alpha = b.activation * 255;

      // Draw the Minimalist Bird Silhouette
      noStroke();
      fill(200, 240, 255, alpha);

      // Body
      ellipse(0, 0, 14, 8);
      // Head
      circle(6, -2, 8);
      // Beak
      triangle(8, -3, 13, -1, 8, 1);

      // Animated Flapping Wing
      let wingY = sin(frameCount * b.flapSpeed + b.flapOffset) * 10;
      triangle(2, 0, -4, wingY, 6, wingY);

      pop();
    }
  }

  // --- 4. Draw the Airplane ---
  push();
  translate(plane.x - cam.x, plane.y);
  noStroke();

  fill(160, 170, 190); triangle(-10, 0, 15, 0, -15, -35); // Back wing
  fill(220, 230, 240); ellipse(0, 0, 90, 22);             // Body
  fill(190, 200, 215); triangle(-30, 0, -45, -25, -20, -5); // Tail
  fill(200, 210, 225); triangle(-15, 5, 25, 5, -25, 45);  // Front wing
  fill(150, 220, 255, 200); arc(25, -4, 25, 16, PI, TWO_PI); // Cockpit

  pop();
}

function drawStars(starArray, parallaxFactor, maxAlpha) {
  noStroke();
  for (let s of starArray) {
    let screenX = (s.x - (cam.x * parallaxFactor)) % viewW;
    if (screenX < 0) screenX += viewW;
    let screenY = s.y;

    let alpha = map(sin(frameCount * 0.05 + s.twinkleOffset), -1, 1, maxAlpha * 0.2, maxAlpha);
    fill(255, 255, 255, alpha);
    circle(screenX, screenY, s.size);
  }
}
```

Would you like me to map the airplane's vertical position (plane.y) to your mouse so you can actively fly up and down to find them, rather than just waiting for them to cross your path?

Prompt 6: Center this frame on the screen, make the canvas black, and add a bit of shadow behind the frame.

Gemini answer:

To get that beautifully framed, museum-like presentation, we need to make changes to both your style.css (to handle the centering and the shadow) and your sketch.js (to change the canvas color to pure black).
Here are the updated files to achieve that look.

Updated style.css

By turning the body into a flexbox, we can perfectly center the canvas vertically and horizontally. I also added a dark gray background to the page so the black canvas and its drop shadow stand out beautifully.
Replace your style.css with this:
CSS

```
html, body {
  margin: 0;
  padding: 0;
  height: 100vh; /* Force the body to take up the full screen height */
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: #1a1a1a; /* Dark gray page background to contrast the black canvas */
}

canvas {
  display: block;
  box-shadow: 0px 20px 50px rgba(0, 0, 0, 0.8); /* Deep shadow behind the canvas */
  border-radius: 4px; /* Optional: A very subtle rounding of the sharp corners */
}
```

Updated sketch.js

The only change here is updating background(12, 16, 35); to background(0); so the sky is pure black, matching your request.
Replace your sketch.js with this:
JavaScript

```
/*
Endless Starry Flight with Minimalist Birds (Framed)
```

```
Goal:
- Infinite starry sky background with parallax depth.
- Airplane automatically flying forward.
- Hidden minimalist birds spawn and illuminate as the plane passes near them.
*/

let plane = { x: 100, y: 240, speed: 3.5 };
let cam = { x: 0, y: 0 };
const viewW = 800, viewH = 480;

let backStars = [];
let midStars = [];
let frontStars = [];

// Array to hold our hidden birds and a tracker for when to spawn the next one
let birds = [];
let nextBirdX = 800;

function setup() {
  createCanvas(viewW, viewH);

  for (let i = 0; i < 150; i++) backStars.push(createStar(1, 2));
  for (let i = 0; i < 100; i++) midStars.push(createStar(2, 3));
  for (let i = 0; i < 40; i++) frontStars.push(createStar(3, 5));
}

function createStar(minSize, maxSize) {
  return {
    x: random(0, viewW),
    y: random(0, viewH),
    size: random(minSize, maxSize),
    twinkleOffset: random(TWO_PI)
  };
}

function draw() {
  background(0); // Pure black canvas

  // --- 1. Update Airplane & Camera ---
  plane.x += plane.speed;
  plane.y = viewH / 2 + sin(frameCount * 0.04) * 25;
  cam.x = plane.x - 200;

  // --- 2. Draw Star Layers ---
  drawStars(backStars, 0.1, 100);
  drawStars(midStars, 0.4, 180);
  drawStars(frontStars, 0.8, 255);

  // --- 3. Procedural Bird Spawning & Discovery ---
  if (cam.x + viewW + 200 > nextBirdX) {
    birds.push({
      x: nextBirdX,
      y: random(80, viewH - 80),
```

```
    speed: random(1, 2),
    flapSpeed: random(0.3, 0.6),
    flapOffset: random(TWO_PI),
    activation: 0,
    discovered: false
  });
  nextBirdX += random(300, 800);
}

// Iterate backwards to safely update and remove off-screen birds
for (let i = birds.length - 1; i >= 0; i--) {
  let b = birds[i];

  // Birds fly forward autonomously
  b.x += b.speed;

  // Garbage collection: Remove birds that are far behind the camera
  if (b.x < cam.x - 200) {
    birds.splice(i, 1);
    continue;
  }

  // Discovery Logic
  let d = dist(plane.x, plane.y, b.x, b.y);

  if (d < 220) {
    b.activation = lerp(b.activation, 1, 0.06);
    b.discovered = true;
  } else {
    let targetAlpha = b.discovered ? 0.2 : 0;
    b.activation = lerp(b.activation, targetAlpha, 0.02);
  }

  // Only draw if the bird is currently visible
  if (b.activation > 0.01) {
    push();
    translate(b.x - cam.x, b.y);

    let alpha = b.activation * 255;

    // Draw the Minimalist Bird Silhouette
    noStroke();
    fill(200, 240, 255, alpha);

    // Body
    ellipse(0, 0, 14, 8);
    // Head
    circle(6, -2, 8);
    // Beak
    triangle(8, -3, 13, -1, 8, 1);

    // Animated Flapping Wing
    let wingY = sin(frameCount * b.flapSpeed + b.flapOffset) * 10;
```

```
      triangle(2, 0, -4, wingY, 6, wingY);

      pop();
    }
  }

  // --- 4. Draw the Airplane ---
  push();
  translate(plane.x - cam.x, plane.y);
  noStroke();

  fill(160, 170, 190); triangle(-10, 0, 15, 0, -15, -35); // Back wing
  fill(220, 230, 240); ellipse(0, 0, 90, 22);             // Body
  fill(190, 200, 215); triangle(-30, 0, -45, -25, -20, -5); // Tail
  fill(200, 210, 225); triangle(-15, 5, 25, 5, -25, 45);  // Front wing
  fill(150, 220, 255, 200); arc(25, -4, 25, 16, PI, TWO_PI); // Cockpit

  pop();
}

function drawStars(starArray, parallaxFactor, maxAlpha) {
  noStroke();
  for (let s of starArray) {
    let screenX = (s.x - (cam.x * parallaxFactor)) % viewW;
    if (screenX < 0) screenX += viewW;
    let screenY = s.y;

    let alpha = map(sin(frameCount * 0.05 + s.twinkleOffset), -1, 1, maxAlpha * 0.2, maxAlpha);
    fill(255, 255, 255, alpha);
    circle(screenX, screenY, s.size);
  }
}
```

Would you like me to map the airplane's vertical position to your mouse now, so you can interactively fly it up and down to find the hidden birds?